

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 1 Part A — Uninformed Search Algorithms

Student Name: _____ Fredda Nyarko _____

Student ID: _____ 92632028 _____

GitHub Repository Link: ___ <https://github.com/fredd-a/ai-course-lab0.git> _____

Date Submitted: _____ 05/06/2026 _____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	yes
Estimated percentage of the work influenced by AI	15%
Did you attach evidence of AI use where applicable?	yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
chatgpt	Understanding how to perform certain tasks and the different ways of performing them.	“how does normalization work in python” “what are some other types of normalization in numpy” “what is a linear regression solution and how is it implemented” “how do i analyse data in python using pandas and numpy, explain it to me “	-Exercise 4: task 2 -Exercise 4: task 3 -Exercise 5: task 1	I used AI to explain the concepts introduced in the lab.

		“how do i fill in missing data, please explain” “how is a pivot table created and how does it work”		

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft		
Prompt history		
Screenshot of AI interaction	yes	
Exported AI conversation		
AI-generated code snippet		
Revised version showing student input		

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	yes
My AI usage does not exceed 25% of the entire work.	yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: _____ fredda_n. _____

Date: _____ 05/06/2026 _____

ChatGPT ▾

 Free offer  Share

how does normalization work in python

Normalization in Python usually means **scaling data to a common range or distribution** so that values can be compared fairly or used effectively in machine learning algorithms.

There are several common types:

1. Min-Max Normalization

Scales values to a range, usually 0 to 1.

Formula:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Example:

 Python

```
data = [10, 20, 30, 40, 50]
```

```
min_val = min(data)
```

Example:

 Python

```
data = [10, 20, 30, 40, 50]
```

```
min_val = min(data)
```

```
max_val = max(data)
```

```
normalized = [(x - min_val) / (max_val - min_val) for x in data]
```

```
print(normalized)
```

Output:

 Python

```
[0.0, 0.25, 0.5, 0.75, 1.0]
```

2. Z-Score Normalization (Standardization)

Transforms data so that:

2. Z-Score Normalization (Standardization)

Transforms data so that:

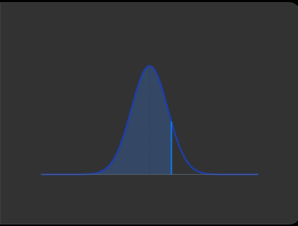
- Mean = 0
- Standard deviation = 1

Formula:

$$z = \frac{x - \mu}{\sigma}$$

$x = 1.2$
 $\mu = 0.0$
 $\sigma = 1.0$

$z = \frac{1.2 - 0}{1} \approx 1.2$
 $\Phi(z) \approx 88.5\%$



Example:

```
Python
import statistics

data = [10, 20, 30, 40, 50]

mean = statistics.mean(data)
std = statistics.stdev(data)
```

```
mean = statistics.mean(data)
std = statistics.stdev(data)

normalized = [(x - mean) / std for x in data]

print(normalized)
```

Output:

```
Python
[-1.2649, -0.6325, 0.0, 0.6325, 1.2649]
```

3. Using NumPy

```
Python
import numpy as np

data = np.array([10, 20, 30, 40, 50])

normalized = (data - data.min()) / (data.max() - data.min())

print(normalized)
```

```
Python
from sklearn.preprocessing import MinMaxScaler

data = [[10], [20], [30], [40], [50]]

scaler = MinMaxScaler()
normalized = scaler.fit_transform(data)

print(normalized)
```

Why normalize data?

Imagine a dataset with:

- Age: 18–70
- Salary: 1,000–100,000

Salary values are much larger and can dominate calculations. Normalization puts them on a similar scale so algorithms like k-NN, neural networks, and gradient descent work better.

If you're studying AI and machine learning, the most important normalization techniques to know are Min-Max Scaling and Z-Score Standardization.

📄 📌 🔍 ⬆️ ↺️ ⋮

NumPy itself doesn't have many named normalization functions built in, but you can implement several common normalization techniques using NumPy operations.

1. Min-Max Normalization

Scales values to a specific range (usually 0 to 1).

```
Python
import numpy as np

x = np.array([10, 20, 30, 40, 50])

normalized = (x - np.min(x)) / (np.max(x) - np.min(x))
print(normalized)
```

2. Z-Score Normalization

Centers data around 0 with a standard deviation of 1.

```
Python
normalized = (x - np.mean(x)) / np.std(x)
print(normalized)
```

3. Decimal Scaling

Moves the decimal point so all values lie between -1 and 1.

```
Python
j = len(str(int(np.max(np.abs(x)))))
normalized = x / (10 ** j)
print(normalized)
```

For `[10, 20, 30, 40, 50]`, this gives:

```
Python
[0.1, 0.2, 0.3, 0.4, 0.5]
```

4. Mean Normalization

Centers data around the mean and scales by the range.

```
Python
normalized = (x - np.mean(x)) / (np.max(x) - np.min(x))
print(normalized)
```

5. L1 Normalization

Makes the sum of absolute values equal to 1.

5. L1 Normalization

Makes the sum of absolute values equal to 1.

 Python

```
normalized = x / np.sum(np.abs(x))  
print(normalized)
```



 Run

Useful in text processing and probability distributions.

6. L2 Normalization (Vector Normalization)

Makes the vector length equal to 1.

 Python

```
normalized = x / np.linalg.norm(x)  
print(normalized)
```



 Run

The norm is:

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

This is very common in machine learning, information retrieval, and computer vision.

7. Max Normalization



7. Max Normalization

Divides every value by the maximum value.

</> Python

📄

▶ Run

```
normalized = x / np.max(x)
print(normalized)
```

Output:

</> Python

📄

▶ Run

```
[0.2, 0.4, 0.6, 0.8, 1.0]
```

Quick Comparison

Method	Typical Range	Purpose
Min-Max	0 to 1	Preserve relative distances
Z-Score	Mean = 0, Std = 1	Statistical analysis, ML
Mean Normalization	Around 0	Center data
Decimal Scaling	-1 to 1	Simple magnitude reduction
L1	Sum = 1	Probabilities, NLP
L2	Length = 1	ML, similarity calculations

L2

Length = 1

ML, similarity calculations

Max Normalization

0 to 1

Simple scaling

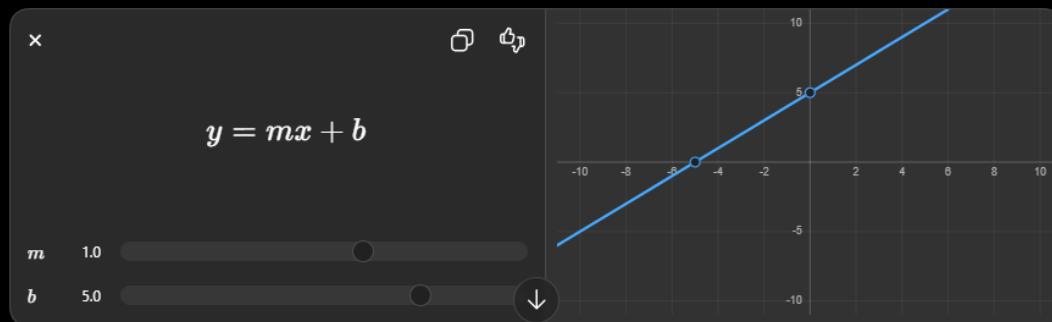
For introductory AI and machine learning courses, **Min-Max**, **Z-Score**, and **L2 normalization** are the ones you'll encounter most often.

     ...  Sources

what is a linear regression solution and how is it implemented

A **linear regression solution** is the process of finding the **best-fitting line** (or hyperplane for multiple variables) that models the relationship between input variables and an output variable.

For a single feature, the model looks like:



where:

- y = predicted output
- x = input feature
- m = slope (weight)
- b = intercept (bias)

The goal is to find values of m and b that minimize the prediction error.

Example

Suppose we have:

Hours Studied	Exam Score
1	50
2	55
3	65
4	70
5	80

Linear regression tries to find a line that best predicts score from hours studied.

How the Solution is Found



Method 1: Closed-Form Solution (Normal Equation)

For multiple features:

$$y = Xw$$

The optimal weights are:

$$w = (X^T X)^{-1} X^T y$$

This directly computes the best parameters.

Implementation with NumPy

 Python



 Run

```
import numpy as np

# Features
X = np.array([1, 2, 3, 4, 5])

# Targets
y = np.array([50, 55, 65, 70, 80])

# Add column of ones for intercept
X_b = np.c_[np.ones(len(X)), X]

# Normal equation
theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

print(theta)
```



how do i analyse data in python using pandas and numpy, explain it to me

Think of **NumPy** as your high-speed calculator engine 🧮 and **Pandas** as your spreadsheet assistant 📊.

Data analysis usually follows a pipeline:

1. Load the data
2. Inspect it
3. Clean it
4. Analyze it
5. Summarize or visualize it

Let's walk through a typical example.

Step 1: Import the Libraries

</> Python



▶ Run

```
import pandas as pd
import numpy as np
```

Step 2: Load Data

Suppose you have a CSV file:



Step 6: Use NumPy for Numerical Analysis

Convert a column into a NumPy array:

 Python

```
scores = df["Score"].to_numpy()
```

 Run

Now use NumPy:

Mean

 Python

```
np.mean(scores)
```

 Run

Standard Deviation

 Python

```
np.std(scores)
```

 Run

Variance

 Python

```
np.var(scores)
```

 Run

Sum



how do i fill in missing data, please explain

Missing data means some values in your dataset are empty, often represented as:

</> Python

NaN



Run

which stands for **Not a Number**.

Example:

Name	Age	Score
Ama	20	85
Kofi	NaN	90
Yaw	21	NaN

Step 1: Find Missing Data

Check if there are missing values

</> Python



Run

A **pivot table** is a tool used to **summarize and analyze data** by grouping rows and calculating statistics such as sums, averages, counts, minimums, or maximums.

Think of it as a data "rotator" 🔄. You take a large table and quickly answer questions like:

- What is the average score per department?
- How many products were sold in each region?
- What is the total revenue per month?

Example Dataset

Student	Department	Score
Ama	IT	80
Kofi	Business	90
Yaw	IT	70
Akosua	Business	85

Creating a Pivot Table in Pandas



Real-World Example

Sales data:

Region	Product	Sales
Accra	Laptop	5000
Kumasi	Laptop	4000
Accra	Phone	3000
Kumasi	Phone	2500

 Python



 Run

```

pivot = pd.pivot_table(
    df,
    values="Sales",
    index="Region",
    columns="Product",
    aggfunc="sum"
)
```

Output:

Region	Laptop		Phone
Accra	5000		3000
Kumasi	4000		2500